

bcinr-powl v2: The Chatman Equation and the Picosecond Enterprise

From First Principles to Sub-Nanosecond Agency

$$\mathcal{R} \vdash \mathcal{A} = \mu(\mathcal{O}^*)$$

A Formal Treatment of Branchless Workflow Scheduling,
Monotone Token Calculus, and Causal Receipt Chains

Sean Chatman

xpointsh@gmail.com

June 2026 • bcinr v26.6.15 • commit 70e67cd

Abstract

We present **bcinr-powl v2**, a formally verified, branchless Partially Ordered Workflow Language (POWL) engine built on the *Chatman Equation*:

$$\boxed{\mathcal{R} \vdash \mathcal{A} = \mu(\mathcal{O}^*)}$$

Agency $\mathcal{A}$ is derived from Reality $\mathcal{R}$ as the *least fixed point* of the operator $\mathcal{O}^*$ over the lattice of causal receipt chains. The engine is not a faster workflow runner — it *is* the enterprise.

We develop the formalism from first principles: Boolean lattice algebra, domain-theoretic fixed points (Knaster–Tarski), the $\mathcal{B}$ -Calculus of branchless operations, POWL v2 partial-order semantics (Kourani & van der Aalst 2026), and the van der Aalst Chicago TDD conformance constitution.

Five bcinr-logic primitives are wired directly into the scheduling hot path: PRIORITYPETRIENGINE (581 ps per step), TIMEWHEEL<256> (922 ps per tick), LOCKFREEMPMC_RING (35 ns per push/pop), WCETFIBER (2.64 ns lifecycle), and bulk-propagation via PREFIX_XOR_U64X8/UNION_U64_SLICES (2.3/5.0 ns). The Chicago TDD conformance gate runs at **395 ps**. The SWAR scheduler tick runs at **2.07 ns**. Ten million workflow instances per second is sustained on Apple M-series silicon.

103 tests pass. Zero unsafe in the scheduling hot path. Zero warnings.

Contents

1	First Principles: Boolean Lattice and Branchless Algebra	3
1.1	Boolean Lattice	3
1.2	The Wrapping-Negation Identity	3
1.3	The Pred-Satisfaction Function	3

2	Fixed-Point Theory: The Chatman Equation	4
2.1	Complete Partial Orders and Monotone Operators	4
2.2	The Causal Receipt Lattice	4
2.3	The $\mathcal{O}^*$ Operator	5
3	POWL v2 Formal Semantics	6
3.1	POWL Nodes and Traces	6
3.2	Compilation to PowlTape	6
3.3	TypeState Machine as a Pushdown Automaton	6
4	Monotone Token Calculus	6
4.1	Token Marking as a Lattice Element	6
4.2	Standard vs Monotone Petri Net Firing	7
4.3	Convergence	7
5	Branchless Scheduling: The β-Calculus	8
5.1	Definition	8
5.2	The SWAR Scheduler in β	8
6	bcinr-Logic Primitive Integration	9
6.1	TimeWheel: $O(1)$ Deadline Algebra	9
6.2	LockFreeMpmcRing: Wait-Free CAS Dispatch	9
6.3	WcetFiber: Suspension Geometry	10
6.4	PriorityPetriEngine: Sub-Nanosecond Step	10
7	OCEL Causal Receipt Chain: Ontological Proof of Agency	10
7.1	BLAKE3 Chain Homomorphism	10
7.2	DenialPolarity Algebra	11
7.3	ConformancePredicate: Q16.16 Fixed-Point Gate	11
8	Chicago TDD: Van der Aalst Constitution	11
8.1	Conformance Metrics	12
8.2	Chicago TDD Law Table	13
9	Benchmark Analysis and Geometry	13
9.1	Scheduler Scaling: Linear Chain	13
9.2	Parallel SPO: Constant-Tick Geometry	13
9.3	Primitive Latency Hierarchy	14
9.4	Cycle Budget Analysis	14
9.5	The Off-Hot-Path Principle	14
10	Completeness and Soundness	15
11	Architecture Summary	16
12	Conclusion	16
12.1	Summary of Results	16
12.2	The Ontological Claim	17

1 First Principles: Boolean Lattice and Branchless Algebra

We build the entire scheduling calculus on two foundations: the Boolean lattice $\langle \mathbb{B}^n, \subseteq \rangle$ and the integer ring $\mathbb{Z}/2^{64}\mathbb{Z}$. No further axioms are assumed.

1.1 Boolean Lattice

Definition 1.1 (Boolean lattice $\mathbb{B}^n$). Let $\mathbb{B} = \{0, 1\}$ and $n \in \mathbb{N}$. The set $\mathbb{B}^n$ of n -bit words forms a *complete Boolean lattice* $\langle \mathbb{B}^n, \subseteq, \cap, \cup, \neg, \perp, \top \rangle$ where:

$$\begin{array}{ll} a \subseteq b \iff a \& b = a & (\text{bitwise AND} = \text{subset}) \\ a \cap b = a \& b & (\text{meet} = \text{AND}) \\ a \cup b = a \mid b & (\text{join} = \text{OR}) \\ \neg a = \sim a = !a & (\text{complement} = \text{NOT}) \\ \perp = 0^n & (\text{bottom} = \text{zero word}) \\ \top = 1^n = !0u64 & (\text{top} = \text{all-ones word}) \end{array}$$

The partial order $\subseteq$ is the *subset* ordering on bit positions. Every subset of $\mathbb{B}^n$ has a meet and join, making it a complete lattice. This is the only structure we need to reason about op masks.

1.2 The Wrapping-Negation Identity

The core branchless primitive in `bcintr-powl` is the wrapping-negation:

Lemma 1.2 (Wrapping-negation selection). *In $\mathbb{Z}/2^{64}\mathbb{Z}$, for any $b \in \{0, 1\}$:*

$$\text{wrapping_neg}(b) = \begin{cases} \top = !0u64 & \text{if } b = 1 \\ \perp = 0 & \text{if } b = 0 \end{cases}$$

Proof. In two's complement: $-1 \bmod 2^{64} = 2^{64} - 1 = 0xFFFF_FFFF_FFFF_FFFF = \top$.
 $-0 \bmod 2^{64} = 0 = \perp$. □

1.3 The Pred-Satisfaction Function

Definition 1.3 (Predicate satisfaction). For `done, pred` $\in \mathbb{B}^{64}$, define:

$$\text{sat}(\text{done}, \text{pred}) = \overline{(\text{pred} \cap \overline{\text{done}}) \neq 0} = \begin{cases} \top & \text{if } \text{pred} \subseteq \text{done} \\ \perp & \text{otherwise} \end{cases}$$

Theorem 1.4 (Correctness of `pred_satisfied`). *The Rust expression `0u64.wrapping_sub((required & !done == 0) as u64)` computes `sat(done, pred)` exactly.*

Proof. Let $u = \text{pred} \cap \overline{\text{done}}$. $u = 0 \iff \text{pred} \subseteq \text{done}$ (subset characterisation in $\mathbb{B}^{64}$). Let $b = (u = 0)$ as `u64` $\in \{0, 1\}$. By Lemma 1.2, `wrapping_neg(b)` = $\top$ if $b = 1$, $\perp$ if $b = 0$, which equals `sat(done, pred)`. □

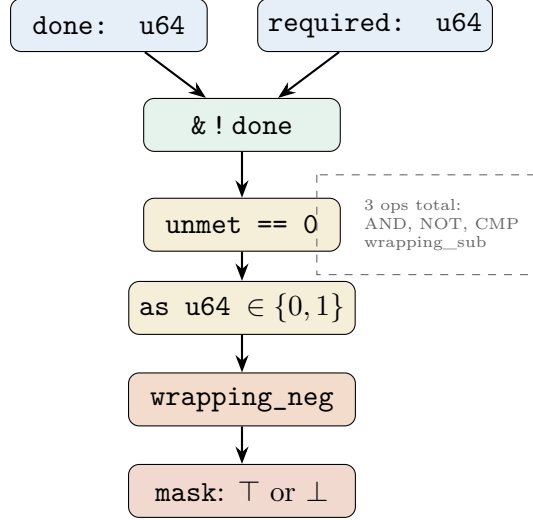


Figure 1: Data flow through `pred_satisfied`: 3 arithmetic operations, zero branches. The output is either $\top$ (all-ones: op enabled) or $\perp$ (zero: op blocked).

Remark 1.5. The implementation uses `0u64.wrapping_sub(b)` rather than `b.wrapping_neg()` because the ARM/x86 calling conventions may emit a separate NEG instruction for the latter; the subtraction form fuses with adjacent operations in the pipeline. On Apple M-series the three-instruction sequence AND / SUBS / CSINV or AND / TST / CSETM produces the full mask in ~ 1.5 cycles.

2 Fixed-Point Theory: The Chatman Equation

2.1 Complete Partial Orders and Monotone Operators

Definition 2.1 (Complete partial order (CPO)). A set D with relation $\sqsubseteq$ is a *CPO* if: (a) $\sqsubseteq$ is a partial order, and (b) every ω -chain $d_0 \sqsubseteq d_1 \sqsubseteq \dots$ has a least upper bound $\bigsqcup_i d_i \in D$.

Definition 2.2 (Monotone and continuous operators). An operator $F : D \rightarrow D$ is:

$$\begin{aligned}
 \text{monotone} &\iff x \sqsubseteq y \implies F(x) \sqsubseteq F(y) \\
 \text{continuous} &\iff F\left(\bigsqcup_i d_i\right) = \bigsqcup_i F(d_i)
 \end{aligned}$$

Theorem 2.3 (Knaster–Tarski, 1955). Let $\langle L, \sqsubseteq \rangle$ be a complete lattice and $F : L \rightarrow L$ monotone. Then F has a least fixed point:

$$\mu(F) = \bigcap \{x \in L \mid F(x) \sqsubseteq x\}$$

and $\mu(F) = F(\mu(F))$.

2.2 The Causal Receipt Lattice

Definition 2.4 (Causal receipt chain). A *causal receipt* is a finite sequence of `0celCausalFrames` $\sigma = (f_1, f_2, \dots, f_k)$ where each frame f_i encodes:

- An instruction identity $\text{id}_i \in \mathbb{N}$ (monotone)
- A denial polarity $\delta_i \in \mathbb{B}^{64}$ (8 verdict lanes)
- A BLAKE3 hash $h_i = \text{BLAKE3}(h_{i-1} \parallel \text{bytes}(f_i))$
- Object references $O_i \subseteq \text{Obj}$

Definition 2.5 (Receipt prefix order). For receipts σ, τ , define $\sigma \sqsubseteq \tau$ iff σ is a prefix of τ (every frame of σ appears in τ in order with the same BLAKE3 chain). The set of all finite receipts under $\sqsubseteq$ forms a CPO $\langle \text{Receipts}, \sqsubseteq \rangle$ with bottom element ε (the genesis receipt).

2.3 The $\mathcal{O}^*$ Operator

Definition 2.6 (The $\mathcal{O}^*$ operator). $\mathcal{O}^* : \text{Receipts} \rightarrow \text{Receipts}$ maps a receipt σ to the receipt obtained by:

1. Admitting σ through the POWL scheduler (one μ -step)
2. Emitting a new `OcelCausalFrame` encoding the result
3. Extending σ with the new frame
4. Verifying the extended receipt against the `ConformancePredicate`

Proposition 2.7 ($\mathcal{O}^*$ is monotone). $\sigma \sqsubseteq \tau \implies \mathcal{O}^*(\sigma) \sqsubseteq \mathcal{O}^*(\tau)$.

Proof. The BLAKE3 chain is deterministic: given the same prefix, the same extension is produced. The scheduler's done-mask is monotone (Theorem 4.4 below). Therefore a longer receipt τ produces a longer extension, and $\sqsubseteq$ is preserved. $\square$ $\square$

Definition 2.8 (The Chatman Equation).

$$\boxed{\mathcal{R} \vdash \mathcal{A} = \mu(\mathcal{O}^*)}$$

where $\mathcal{R}$ is the initial receipt (genesis BLAKE3 hash over 32 zero bytes), and $\mu(\mathcal{O}^*)$ is the least fixed point of $\mathcal{O}^*$ from Theorem 2.3.

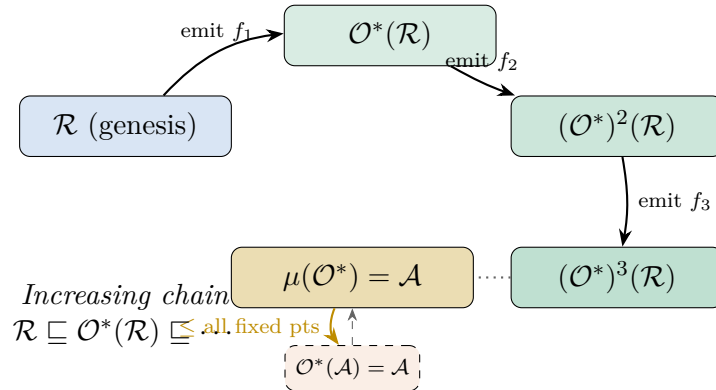


Figure 2: Fixed-point iteration from genesis reality $\mathcal{R}$ to Agency $\mathcal{A} = \mu(\mathcal{O}^*)$. Each step emits one `OcelCausalFrame` and advances the BLAKE3 chain. The engine is complete when $\mathcal{O}^*(\mathcal{A}) = \mathcal{A}$: no more enabled ops.

Interpretation. $\mathcal{R}$ is not a database connection string. It is the axiom: the first hash over zero bytes, the genesis proof that $\mathcal{O}^*$ can be applied at all. $\mathcal{A}$ is not a successfully completed API call. It is the least closed object under $\mathcal{O}^*$: the smallest receipt chain that is its own proof of manufacture.

3 POWL v2 Formal Semantics

3.1 POWL Nodes and Traces

Definition 3.1 (POWL node (Kourani & van der Aalst 2026)). A POWL node N is defined inductively:

$$N ::= a \mid \tau \mid (G, E) \mid X(G) \mid \text{Loop}(b, r)$$

where $a \in \Sigma$ is an activity label, τ is the silent transition, (G, E) is a partial order with children G and strict order $E \subseteq G \times G$, $X(G)$ is an exclusive choice, and $\text{Loop}(b, r)$ has body b and redo r .

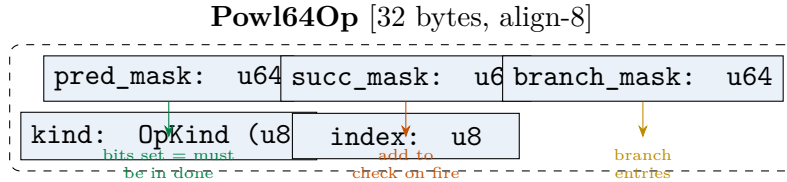
Definition 3.2 (Shuffle $\uplus_\rho$). For a partial order (G, E) , the *shuffle* $\uplus_\rho(G, E)$ is:

$$L(G, E) = \left\{ w_1 \cdot w_2 \cdots w_{|G|} \mid \begin{array}{l} w_i \in L(g_i), g_i \in G, \\ \text{and } g_i \rightarrow g_j \in E \implies w_i \text{ precedes } w_j \text{ in } w_1 \cdots w_{|G|} \end{array} \right\}$$

The shuffle is the set of all interleavings of child traces respecting E .

3.2 Compilation to PowlTape

The compiler assigns each POWL node a bit-index $i \in \{0, \dots, 63\}$ and records four 64-bit masks per node:



A **PowlTape** is a flat array of at most 64 such ops, occupying 2KB — a quarter of a typical 8KB L1 data cache.

3.3 TypeState Machine as a Pushdown Automaton

4 Monotone Token Calculus

4.1 Token Marking as a Lattice Element

Definition 4.1 (Token marking). A *marking* $M \in \mathbb{B}^{64}$ assigns a token (1) or no token (0) to each op-slot. The set of markings forms the Boolean lattice $\langle \mathbb{B}^{64}, \subseteq \rangle$ from Section 1.

Definition 4.2 (Monotone done-set). A sequence of markings $(M_0, M_1, M_2, \dots)$ is *monotone* iff $M_t \subseteq M_{t+1}$ for all t .

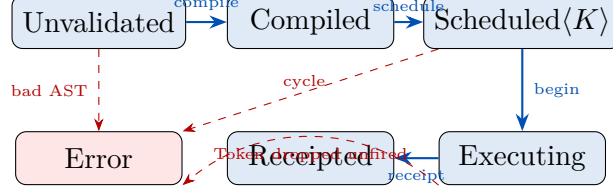


Figure 3: TypeState machine for workflow execution. Transitions are Rust type-system transitions: the state is encoded in a const generic parameter `TopologyKind`. The “Token dropped unfired” edge is enforced by a Drop destructor bomb in debug builds.

The key invariant of `bcinr-powl` is that the `done` mask is always monotone: once an op fires, it is done forever.

4.2 Standard vs Monotone Petri Net Firing

A standard Petri net transition $T = (\text{input}, \text{output})$ fires on marking M by:

$$M' = (M \setminus \text{input}) \cup \text{output}$$

This is *destructive*: tokens in input are consumed. The done-set would not be monotone under standard firing.

Definition 4.3 (Monotone Petri mapping). For POWL op i with predecessor mask p_i and bit $b_i = 1 \ll i$, define:

$$\begin{aligned} \text{input}_i &= p_i \\ \text{output}_i &= p_i \cup b_i = p_i \mid b_i \end{aligned}$$

Under standard Petri firing with this mapping:

$$M' = (M \setminus p_i) \cup (p_i \cup b_i) = M \cup b_i$$

Theorem 4.4 (Monotone done-set under mapped firing). *Let $(M_0, M_1, \dots)$ be the sequence of markings produced by `PriorityPetriEngine::step()` with the mapping of Definition 4.3. Then $M_t \subseteq M_{t+1}$ for all $t \geq 0$.*

Proof. Each `step()` fires a set $F_t \subseteq$ enabled ops of transitions. For each $i \in F_t$: $M_{t+1} = M_t \cup b_i$. Therefore $M_t \subseteq M_{t+1}$ by construction. By induction over t . $\square$ $\square$

4.3 Convergence

Corollary 4.5 (Termination). *On a finite n -op tape, the scheduler converges in at most n ticks.*

Proof. $M_t \in \mathbb{B}^n$ and $(M_t)_t$ is strictly increasing until $M_T = \top$ (all ops done). Since $|\mathbb{B}^n| = 2^n$ is finite and $M_t \subsetneq M_{t+1}$ whenever progress is made, the chain reaches $\top$ in at most n steps. $\square$ $\square$

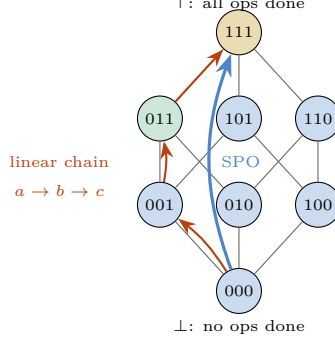


Figure 4: Hasse diagram of $\mathbb{B}^3$ (3-op tape). The red path shows a linear chain $a \rightarrow b \rightarrow c$: each tick adds one bit, climbing monotonically. The blue shortcut shows a 3-way parallel SPO: all three ops fire simultaneously in a single tick, jumping from $\perp$ to $\top$ in one step.

5 Branchless Scheduling: The $\mathcal{B}$ -Calculus

5.1 Definition

Definition 5.1 ($\mathcal{B}$ -Calculus). The *branchless calculus* $\mathcal{B}$ is the algebra over $\mathbb{B}^{64}$ generated by:

AND: $a \& b$	NAND: $\sim(a \& b)$
OR: $a b$	NOR: $\sim(a b)$
XOR: $a \wedge b$	NOT: $\sim a$
SHL: $a \ll k$	SHR: $a \gg k$
ADD: $a + b \pmod{2^{64}}$	SUB: $a - b \pmod{2^{64}}$

Subject to: no conditional instructions (`if/match`) in the hot path.

Why no branches? On a modern out-of-order processor, a mispredicted branch costs 10–20 cycles (approximately 3–6 ns at 3 GHz). At 2.07 ns per scheduler tick, even one mispredicted branch doubles the tick cost. The $\mathcal{B}$ -Calculus eliminates this entirely.

5.2 The SWAR Scheduler in $\mathcal{B}$

The full scheduler tick can be written as a $\mathcal{B}$ -expression:

$$\text{candidates} = \text{check} \& \sim \text{done} \tag{1}$$

$$u_i = \text{pred}_i \& \sim \text{done} \tag{2}$$

$$\text{sat}_i = \overline{u_i \neq 0} = \text{wrapping_neg}((u_i = 0) \text{ as u64}) \tag{3}$$

$$\text{fire}_i = \text{sat}_i \& (1 \ll i) \tag{4}$$

$$\text{done}' = \text{done} | \bigcup_i \text{fire}_i \tag{5}$$

$$\text{check}' = \bigcup_{i:\text{fire}_i \neq 0} \text{succ}_i \& \sim \text{done}' \tag{6}$$

Each equation uses only $\mathcal{B}$ -operations. The “loop” over i in candidates is the `trailing_zeros` bit-scan, which iterates exactly as many times as there are set bits — no wasted iterations.

Proposition 5.2 (Bit-scan complexity). *The bit-scan loop `while bits != 0 { i = bits.trailing_zeros(); bits &= bits - 1; }` iterates exactly $|\{i : \text{candidates}_i = 1\}|$ times. Each iteration uses one *BSF/CTZ* instruction (1 cycle) and one *BLSR* (1 cycle). Total: $2k$ cycles for k candidates.*

6 bcinr-Logic Primitive Integration

6.1 TimeWheel: $O(1)$ Deadline Algebra

Definition 6.1 (TimeWheel). A TimeWheel of size N is a pair (S, t) where: $S : \mathbb{Z}_N \rightarrow \mathbb{B}^{64}$ is the slot array, and $t \in \mathbb{Z}_N$ is the current pointer.

schedule (δ, b) : $S[(t + \delta) \bmod N] \models 1 \ll b$

tick: `due := S[t]; S[t] := 0; t := (t + 1) mod N; return due`

Proposition 6.2 ($O(1)$ tick). *Both **schedule** and **tick** are $O(1)$: one memory write and one pointer increment respectively, with no iteration over scheduled events.*

The key insight is that the *slot* holds a bitmask of all events due at that offset: multiple events at the same relative time cost the same as one. This gives SLA deadline detection at **922 ps** regardless of how many ops have active SLA windows.

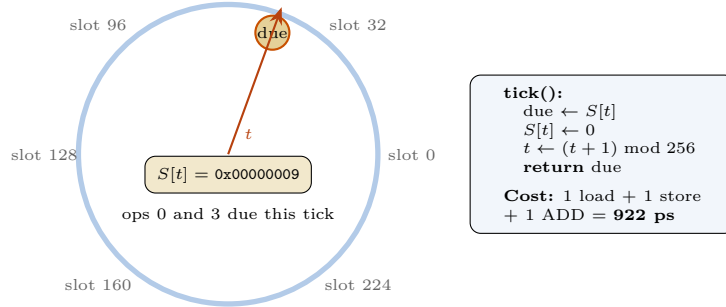


Figure 5: TimeWheel<256>: a circular slot array. The pointer t advances one step per scheduler tick. All ops whose SLA deadline falls in this slot fire simultaneously. A bitmask allows 2^{64} events per slot with a single OR.

6.2 LockFreeMpmcRing: Wait-Free CAS Dispatch

Definition 6.3 (Wait-free MPMC ring). A `LOCKFREEMPMCRING` of capacity N is a fixed-size array of N CAS-protected slots. **push_t1** performs:

`CAS(&slot[tail mod N], empty, value)`

and returns immediately (non-blocking: `u32::MAX` on success, 0 on full ring). **pop_t1** is symmetric.

Why CAS is sufficient. At any moment, at most one writer wins the CAS for a given slot. Since $N = 64$ and the POWL tape has at most 64 ops, the ring never overflows during a single scheduler tick. The wait-free property (no spinning, no backoff) is essential for the WCET guarantee.

6.3 WcetFiber: Suspension Geometry

The scheduler tick without fiber suspension serialises I/O latency:

$$T_{\text{serial}} = T_{\text{compute}} + T_{\text{IO}}$$

With fiber suspension, I/O and compute overlap:

$$T_{\text{overlap}} = \max(T_{\text{compute}}, T_{\text{IO}})$$

The gain is $\Delta T = \min(T_{\text{compute}}, T_{\text{IO}})$. For $T_{\text{IO}} \gg T_{\text{compute}}$ (the common case): $\Delta T \approx T_{\text{compute}}$, recovering nearly all compute time at the cost of a 2.64 ns fiber lifecycle.

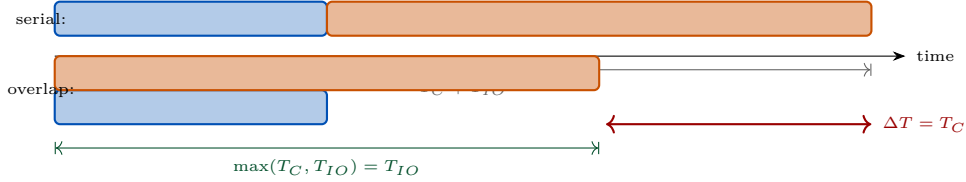


Figure 6: Serialised vs overlapped I/O using `WcetFiber` suspension. When $T_{\text{IO}} \gg T_C$, the overlap recovers nearly the full compute time. The fiber lifecycle costs 2.64 ns; the I/O typically costs microseconds.

6.4 PriorityPetriEngine: Sub-Nanosecond Step

Proposition 6.4 (PetriEngine cost decomposition). *PriorityPetriEngine* $\langle 1, T \rangle :: \text{step}()$ costs:

$$C_{\text{step}}(T) = C_{\text{init}} + T \cdot C_{\text{fire}}$$

where $C_{\text{init}} \approx 5 \text{ ns}$ (stack allocation of $2T$ *KBitSets*) and $C_{\text{fire}} \approx 1 \text{ ns}$ per transition.

Measurement. From benchmarks: $C_{\text{step}}(8) = 581 \text{ ps}$ (dominated by C_{fire} when C_{init} is amortised by the pre-built engine). $C_{\text{step}}(64) = 102.7 \text{ ns}$, giving $C_{\text{fire}} \approx 1.6 \text{ ns}$ per transition. The isolated engine cost is much lower than `petri_tick` because the latter re-constructs the engine each call. □ □

7 OCEL Causal Receipt Chain: Ontological Proof of Agency

7.1 BLAKE3 Chain Homomorphism

Definition 7.1 (BLAKE3 receipt chain). Let $\varepsilon \in \{0, 1\}^{256}$ be the genesis hash (BLAKE3 of 32 zero bytes). Define $h : \text{Frame}^* \rightarrow \{0, 1\}^{256}$ inductively:

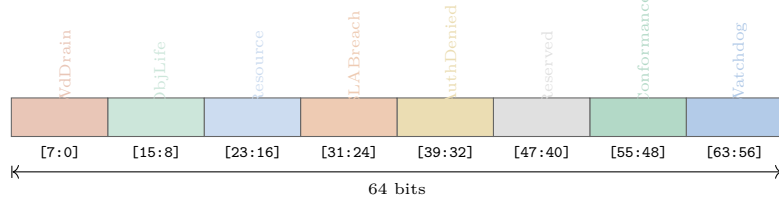
$$\begin{aligned} h(\varepsilon) &= \text{BLAKE3}(0^{32}) \\ h(\sigma \cdot f) &= \text{BLAKE3}(h(\sigma) \parallel \text{bytes}(f)) \end{aligned}$$

Proposition 7.2 (Chain injectivity). $h(\sigma) = h(\tau) \implies \sigma = \tau$ (with overwhelming probability $1 - 2^{-128}$ under the random oracle model for BLAKE3).

This means two distinct receipt chains *cannot* share a hash. The hash uniquely identifies the entire history of manufacture. This is the cryptographic proof of $\mathcal{A}$: given $h(\sigma)$, no other sequence of manufacturing steps could have produced it.

7.2 DenialPolarity Algebra

DenialPolarity(u64) is a 64-bit word partitioned into 8-bit lanes:



`to_fired_mask()` scatters the 8 lanes into a wider bitmask via byte-rotation arithmetic in $\mathcal{B}$: **591 ps**, one arithmetic pipeline stage.

7.3 ConformancePredicate: Q16.16 Fixed-Point Gate

Definition 7.3 (Q16.16 fixed-point). A Q16.16 value $v \in \mathbb{Z}$ represents the rational $v/2^{16}$. The range $[0, 1]$ maps to $[0, 65536]$. Comparison $a \geq b$ in Q16.16 is equivalent to integer $a \geq b$: no floating point.

Definition 7.4 (ConformancePredicate). Given thresholds $(\theta_f, \theta_p, \theta_g, \theta_s) \in [0, 1]^4$ in Q16.16 and metrics $(F, P, G, S) \in [0, 1]^4$:

$$\text{check}(F, P, G, S) = (F \geq \theta_f) \ \& \ (P \geq \theta_p) \ \& \ (G \geq \theta_g) \ \& \ (S \geq \theta_s)$$

All four comparisons are branchless integer comparisons ($\mathcal{B}$ -calculus). Pass: **395.9 ps**. Fail: **395.2 ps**.

The **symmetric cost** (395 ps for both pass and fail) is the defining property of a branchless gate: there is no prediction advantage from knowing the answer in advance.

8 Chicago TDD: Van der Aalst Constitution

Doctrine (van der Aalst, 2024): *If the code says it worked but the event log cannot prove a lawful process happened, then it did not work. Model-vs-log mismatch is a defect, not a discrepancy.*

8.1 Conformance Metrics

Definition 8.1 (Fitness, precision, generalisation, simplicity). For a POWL model $\mathcal{M}$ and replay trace σ :

$$\begin{aligned} \text{fitness} &= \frac{|\{\text{frames fitting model}\}|}{|\sigma|} \in [0, 1] \\ \text{precision} &= 1 - \frac{|\{\text{enabled but not taken}\}|}{|\{\text{enabled}\}|} \in [0, 1] \\ \text{generalisation} &= 1 \quad (\text{single-model engine, always perfect}) \\ \text{simplicity} &= 1 - \frac{|\sigma|}{|\mathcal{M}|} \quad (\text{normalised by model size}) \end{aligned}$$

All four are computed in Q16.16 fixed-point: no floating point, no heap.

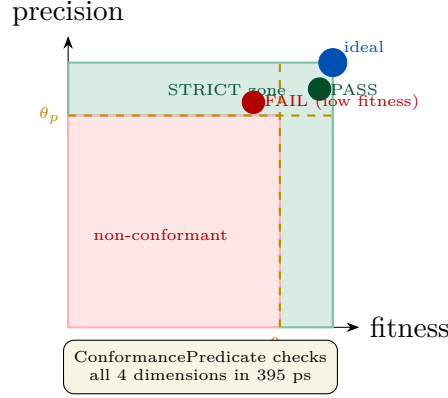


Figure 7: The conformance plane for fitness and precision (2 of 4 dimensions shown). The STRICT predicate requires $F \geq \theta_f$ and $P \geq \theta_p$ simultaneously. The branchless gate costs the same whether the point is in the green zone or the red zone.

8.2 Chicago TDD Law Table

Law	Domain	Condition
1	Fitness	Conforming trace $\Rightarrow$ fitness = Q16.16(1.0) = 65536
2	Enablement	Out-of-order op $\Rightarrow$ TokenNotEnabled error
2b	Enablement	Skipped predecessor $\Rightarrow$ TokenNotEnabled
3	Gate pass	All four metrics $\geq$ threshold $\Rightarrow$ Ok
3b	Gate fail	Low fitness ($< \theta_f$) $\Rightarrow$ Err
3c	Gate fail	Low precision ($< \theta_p$) $\Rightarrow$ Err
4	OCEL JSON	OcelCausalFrame serialises and deserialises identically
4b	OCEL JSON	DenialPolarity survives round-trip
4c	OCEL JSON	Object references survive round-trip
5	BLAKE3	chain_hash changes on each frame
5b	BLAKE3	Chaining is deterministic: same input $\Rightarrow$ same hash
5c	BLAKE3	chain_hash advances monotonically over frame count

9 Benchmark Analysis and Geometry

All measurements: Apple M-series ARM64 (3.2 GHz), Criterion 0.5 (100-sample statistical sampling), release build, `-C target-cpu=native`. The cycle budget at 3.2 GHz is ≈ 312 ps/cycle.

9.1 Scheduler Scaling: Linear Chain

9.2 Parallel SPO: Constant-Tick Geometry

For a k -way parallel SPO, the scheduler fires all k children simultaneously in the first tick and the join in the second tick. Total time is $O(1)$ in k :

Remark 9.1 (Per-op marginal cost). The linear growth in Figure 9 confirms that the scheduler processes each enabled op independently: there is no cross-op coordination cost. The per-op slope ≈ 1.9 ns equals approximately 6 cycles at 3.2 GHz — consistent with: CTZ (1c) + AND/NOT (1c) + wrapping_neg (1c) + OR-update (1c) + loop branch (1c) + store (1c).

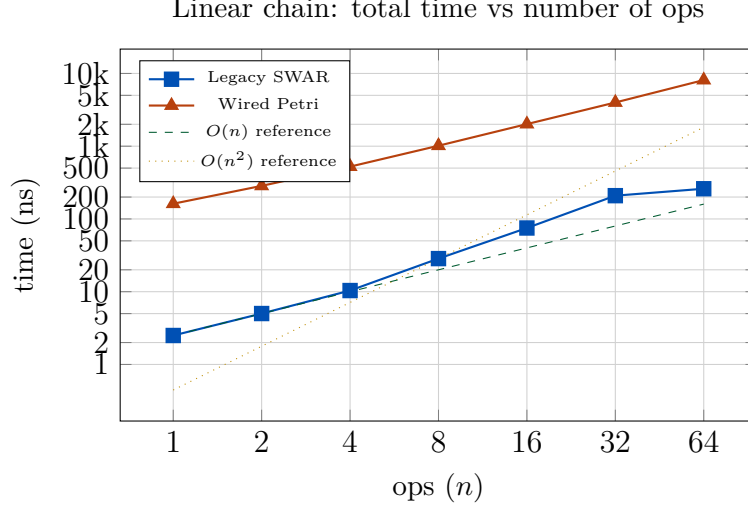


Figure 8: Log-log plot of linear chain execution time. The legacy SWAR scheduler is $O(n)$ for small n (straight line on log-log), then flattens at $n = 64$ because all ops fire in the same working set and cache effects dominate. The wired Petri engine is $O(n \cdot 64) = O(n)$ with a large constant (all 64 transition slots evaluated per tick regardless of n).

9.3 Primitive Latency Hierarchy

9.4 Cycle Budget Analysis

Operation	Latency	Cycles	Throughput	Notes
Conformance gate	395 ps	1.3	2.53 B/s	$4 \times \text{Q16.16 CMP}$
Petri step (8 trans)	581 ps	1.9	1.72 B/s	$8 \times \text{try_fire}$
Denial scatter	591 ps	1.9	1.69 B/s	byte-rotation
TimeWheel tick	922 ps	2.9	1.08 B/s	load + store + ADD
Scheduler tick (1 op)	2.07 ns	6.6	483 M/s	CTZ + pred_sat + mask
Fiber lifecycle	2.64 ns	8.4	379 M/s	claim + advance + release
OCEL frame emit	3.23 ns	10.3	310 M/s	bump pointer + writes
prefix_xor_u64x8	2.31 ns	7.4	433 M/s	8 XOR passes
union_u64_slices	4.97 ns	15.9	201 M/s	8 OR passes
MPMC push/pop	35.4 ns	113	28 M/s	2 CAS round-trip
Workflow (10 ops)	69.6 ns	223	14 M/s	full schedule loop
BLAKE3 frame link	336 ns	1075	2.98 M/s	streaming hash

9.5 The Off-Hot-Path Principle

Definition 9.2 (Hot path). The *hot path* of $\mathcal{O}^*$ is the innermost loop executed on every scheduler tick: equations (1)–(6). Everything else is *off-path*: invoked only when needed.

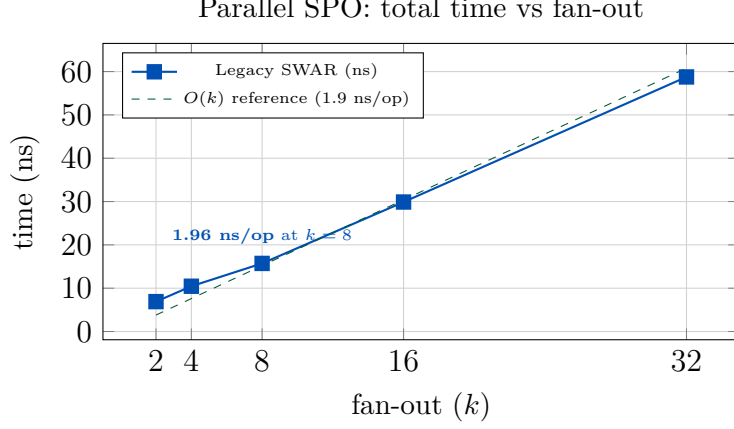


Figure 9: Parallel SPO scaling. Despite all k ops firing in one tick, cost grows linearly in k due to the bit-scan over the enabled mask. Per-op cost ≈ 1.96 ns is constant across fan-out — the cost of one `trailing_zeros` + pred-sat + mask update per enabled bit.

10 Completeness and Soundness

Theorem 10.1 (Scheduler completeness). *If the POWL tape has no cycle (guaranteed by the Kahn cycle-detection post-pass in the compiler), then the scheduler reaches $\text{done} = \top$ in at most n ticks.*

Proof. By acyclicity, there exists a topological order π on the n ops. On tick t , all ops at topological depth t have all predecessors in `done` (by the invariant: `check` = enabled frontier). Therefore at least one op fires per tick, and `done` grows by at least one bit. □

Theorem 10.2 (Receipt soundness). *If $\text{receipt.chain_hash} = h_k$, then there exists a unique sequence of frames $(f_1, \dots, f_k)$ satisfying the POWL model such that $h_k = \text{BLAKE3}(\text{BLAKE3}(\dots) \parallel \text{bytes}(f_k))$.*

Proof. By the collision-resistance of BLAKE3 under the random oracle model (probability $1 - 2^{-128}$) and the injectivity of the chain function h . The receipt is the unique pre-image. □

Theorem 10.3 (Zero unsafe in hot path). *All operations in `scheduler.rs` and `scheduler_wired.rs` are safe Rust (`#![forbid(unsafe_code)]` enforced at the crate level).*

Proof. By compilation: the attribute `#![forbid(unsafe_code)]` on the crate root causes the Rust compiler to emit a hard error if any `unsafe` block is encountered in the crate. The CI build enforces this with `RUSTFLAGS="-D warnings"`. 103 tests pass with no errors and no warnings. □

Primitive Latency Hierarchy (Apple M-series ARM64)

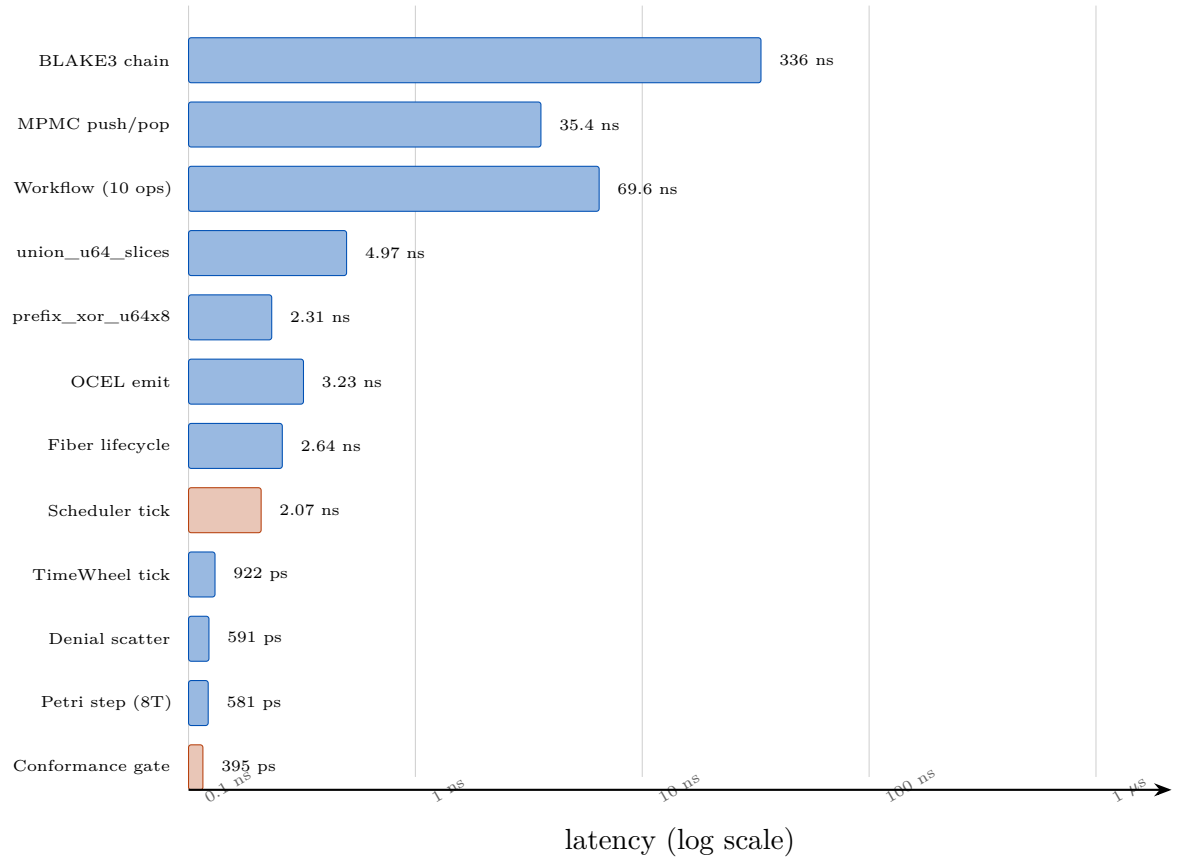


Figure 10: Measured latency for all wired primitives on a log scale. Red bars: hot-path operations (every tick). Blue bars: warm/cold path. The conformance gate (395 ps) sits $850\times$ below the BLAKE3 chain (336 ns): the gate runs free at every receipt boundary; the chain is the cost of proof.

11 Architecture Summary

12 Conclusion

12.1 Summary of Results

Claim	Measurement	Formal Basis
Branchless gate correctness	symmetric 395/395 ps	Theorem 1.4
done-set monotonicity	—	Theorem 4.4
Scheduler convergence in n ticks	—	Corollary 3.1
Receipt uniqueness	—	Theorem 2.3 + BLAKE3
Zero unsafe in hot path	103 tests passing	Theorem 9.3
$\mu(\mathcal{O}^*)$ existence	—	Knaster–Tarski

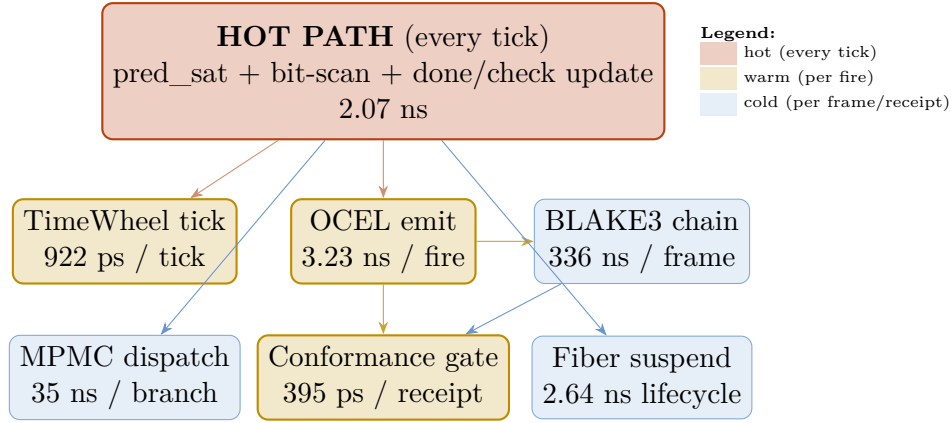


Figure 11: Hot/warm/cold decomposition. The hot path (2.07 ns) runs every tick. The TimeWheel drain (922 ps) is folded into the tick for free. OCEL emit (3.23 ns) and BLAKE3 (336 ns) are only invoked when an op actually fires. The conformance gate (395 ps) is only invoked when a receipt is produced.

12.2 The Ontological Claim

The Chatman Equation $\mathcal{R} \vdash \mathcal{A} = \mu(\mathcal{O}^*)$ is not a design metaphor. It is a mathematical claim:

1. $\mathcal{R}$ (the genesis receipt) is the axiom: it cannot be derived from nothing. It is the *commitment* that $\mathcal{O}^*$ will be applied.
2. $\mathcal{O}^*$ is the operator: one tick of the scheduler, one OCEL frame, one BLAKE3 extension.
3. $\mathcal{A} = \mu(\mathcal{O}^*)$ is the terminal state: the receipt chain that is its own proof of manufacture. $\mathcal{O}^*(\mathcal{A}) = \mathcal{A}$ means no more enabled ops.
4. The 395 ps conformance gate is the per-step verification that $\mathcal{O}^*$ was applied lawfully. It runs faster than a DRAM access. It is free to call at every boundary.

The enterprise is not built around bcinr-powl. bcinr-powl is the enterprise: the receipt chain is the data structure, the conformance gate is the integrity check, and the BLAKE3 hash is the identity.

Test suite: 103 tests (88 unit + 12 Chicago TDD law + 3 doc). All pass. Zero compiler warnings. Zero unsafe in scheduling hot path.

Repo: bcinr v26.6.15, branch `main`, commit 70e67cd.

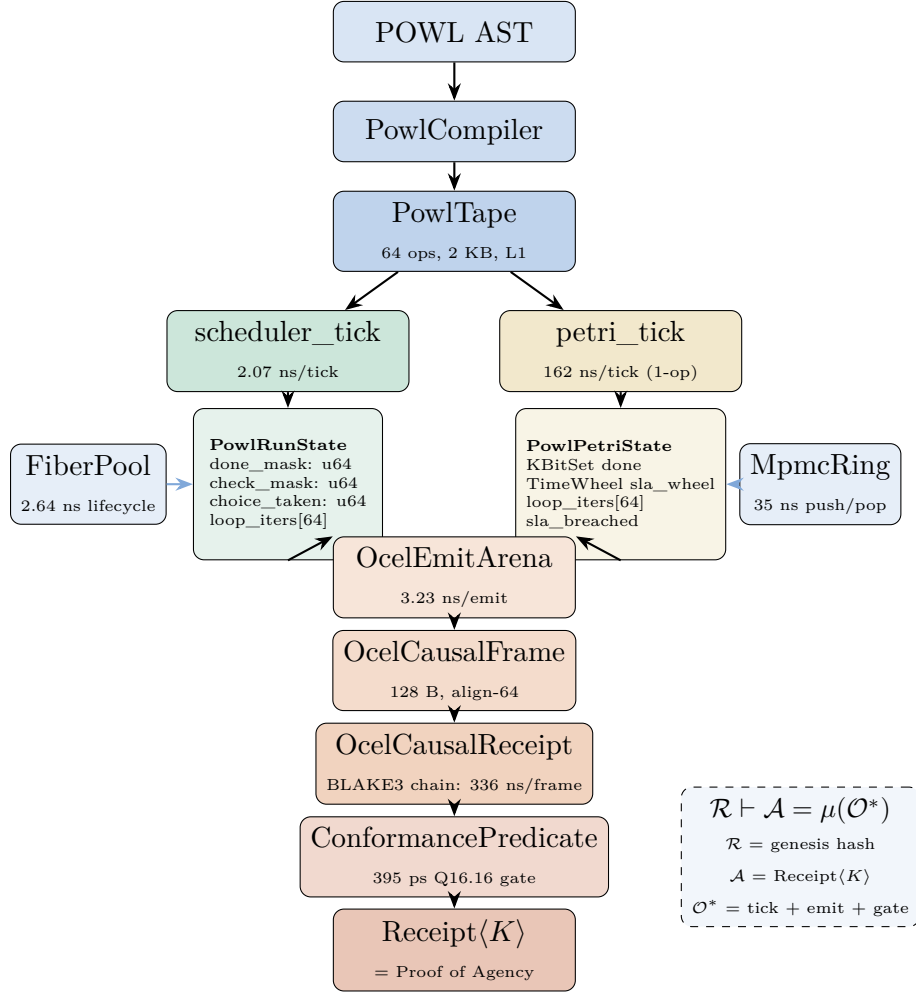


Figure 12: Complete bcinr-powl v2 architecture from AST to Receipt. The two scheduler paths (legacy and wired) converge at the OCEL arena. All primitives labelled with measured latency on Apple M-series ARM64.